SmartFin

A personal finance copilot for Thailand that reads your PromptPay slips and reasons over your own spending — instead of asking you to type it in.

Thaw Zin Moe Myint
Student

Flutter · Go · MongoDB · Gemini
Stack

Alpha Test
Milestone

Thai users already have a record of every payment. It is trapped in their photo gallery.

1 · MANUAL ENTRY FAILS

Budgeting apps depend on the user typing each expense. People abandon this within weeks, so the data is incomplete exactly when advice would matter.

2 · NO BANK AGGREGATION

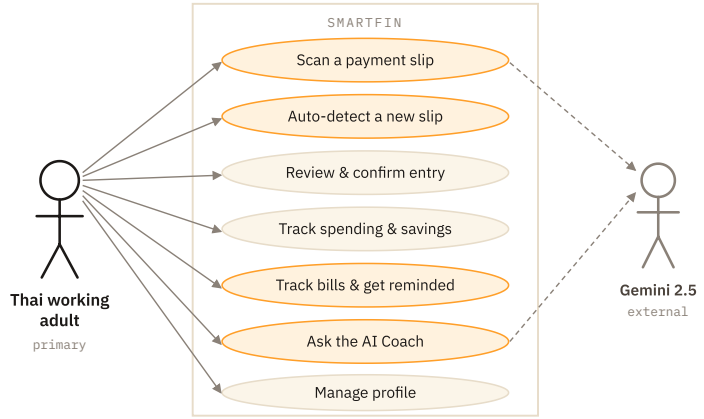
Thailand has no consumer open-banking API. Services like Plaid do not cover Thai retail banks, so automatic transaction import is not available to a student project.

3 · GENERIC ADVICE

Chatbot finance tools answer from general knowledge. They cannot say what *this* user spent on food this month, because they never see the data.

The opening. Every PromptPay, K PLUS, SCB EASY, Krungthai NEXT and Bangkok Bank transfer ends in a confirmation slip the user screenshots. The record already exists — it is simply an image. SmartFin turns that image into structured data, then lets an AI agent reason over it.

Who it is for, and what they can do



Highlighted ellipses are the differentiating use cases — the demo follows them.

PRIMARY USER

A Thai working adult or student who pays by **PromptPay several times a day**, keeps every confirmation slip in their gallery, and has abandoned at least one budgeting app.

DESIGN CONSEQUENCES

- Currency is **integer Thai Baht (฿)** end-to-end — no floats, no decimals.
- Slips print Buddhist-era years: 2569 → 2026 conversion is required, not optional.
- Merchant names arrive in Thai script and are transliterated on extraction.
- Month boundaries and bill due dates are pinned to `UTC+07:00`, not server or phone time.

What the project commits to — and what it deliberately excludes

IN SCOPE

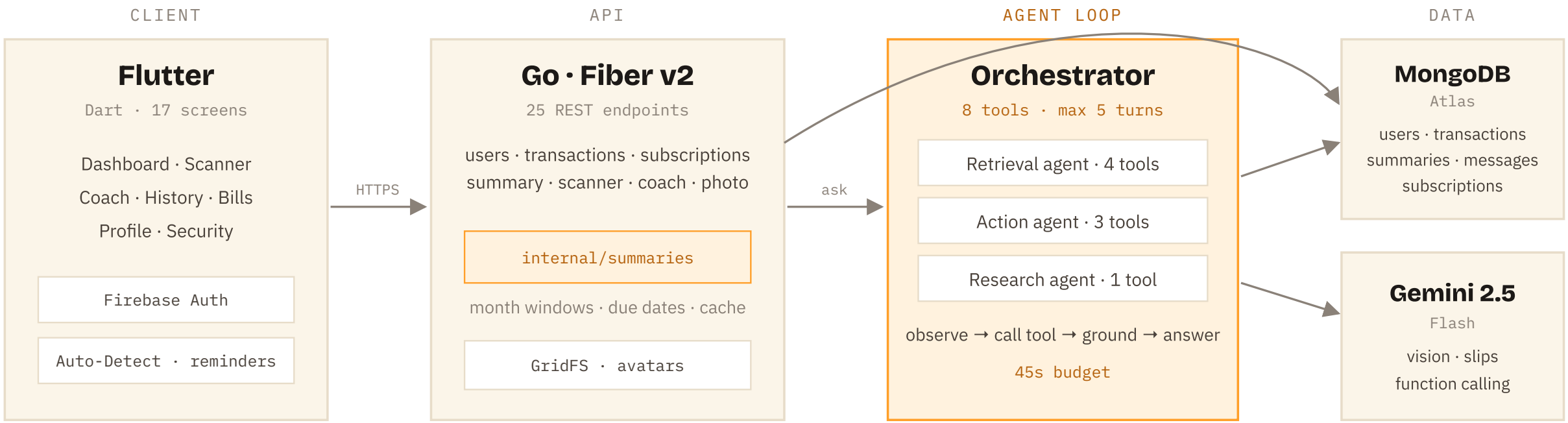
- **Slip & receipt capture** — manual scan and Auto-Detect, both extracted by a vision model.
- **Transaction ledger** — create, list, edit, delete; manual entry as a fallback path.
- **Monthly budgeting** — income, budget limit, top spending categories, per-month summaries.
- **Agentic AI Coach** — answers grounded in the user's own records, and able to act on them.
- **Savings goals** — set by the user or by the coach, with progress derived from actual months.
- **Subscriptions & bills** — tracked by hand, paid in one tap, reminded on the device.
- **Account management** — profile, photo, credentials, account deletion.

OUT OF SCOPE

- **Direct bank integration** — no consumer open-banking API exists in Thailand; the slip *is* the integration.
- **Moving money** — SmartFin never initiates a payment or transfer. It is read-and-advise only.
- **Investment or tax advice** — the coach redirects to general financial health.
- **Multi-currency** — Thai Baht only, which is what makes integer money safe.
- **Multi-user or shared budgets** — one account, one person.

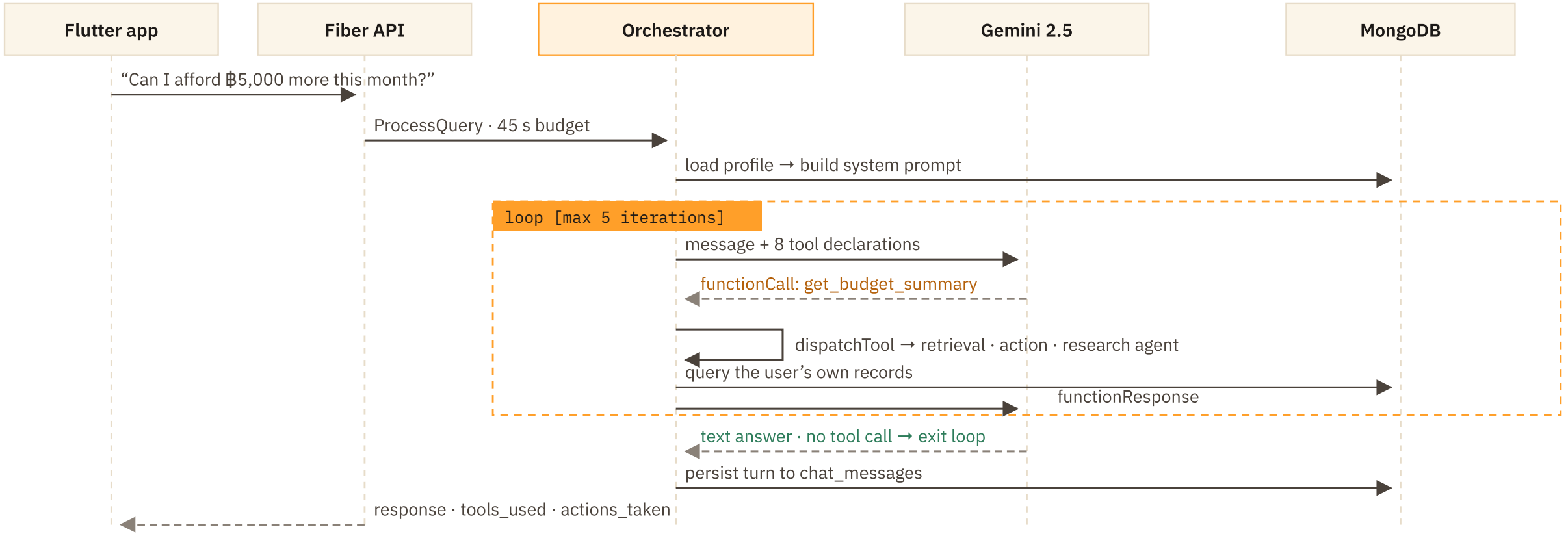
Why the boundary sits here. Every excluded item either requires a commercial banking agreement or moves the project from *advice* to *financial transactions*, which carries a regulatory burden a one-semester project cannot honestly meet.

Three tiers, one agent loop



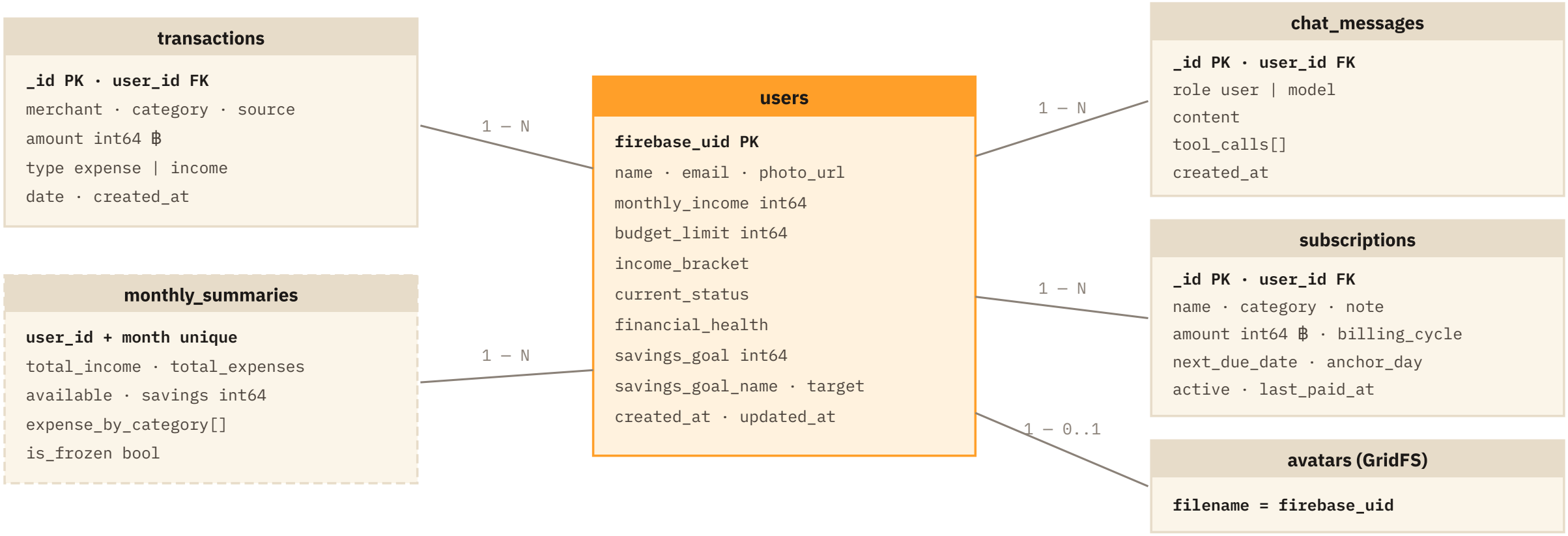
The coach never queries Mongo directly — every read and write goes through a declared tool, which is what makes its actions auditable.

How one question becomes a grounded answer



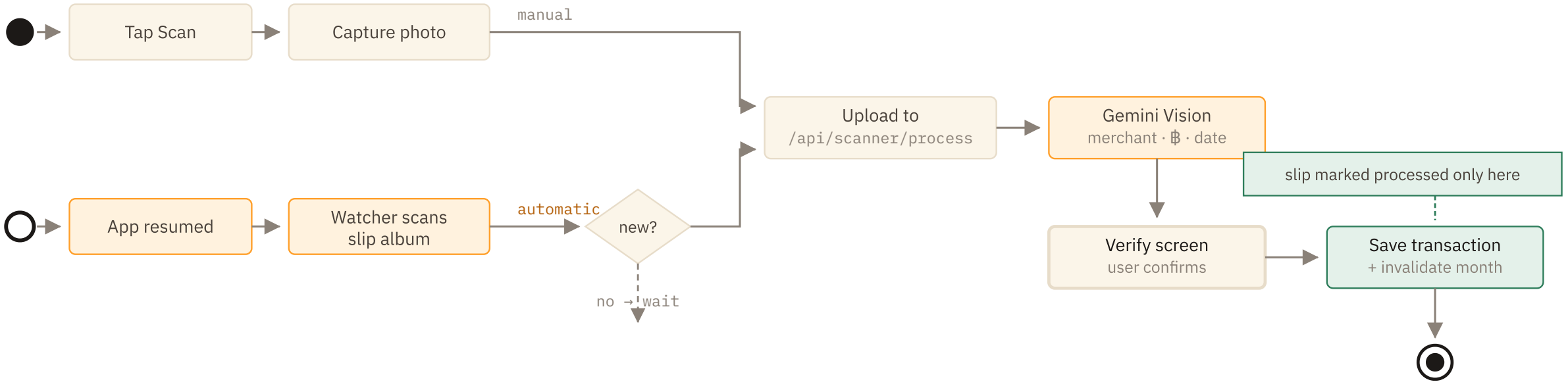
The model is never given database access — it can only request one of eight declared tools, which is what makes every action it takes reviewable.

Five collections, keyed by one identity



Every collection is keyed by the Firebase UID *string*, not a Mongo ObjectID. **monthly_summaries** is dashed because it is derived — a cache rebuilt from transactions, never a source of truth. **subscriptions** stores each due date as a Thai calendar day plus an anchor day, so a bill due on the 31st returns to the 31st after February.

Two ways in, one path to a saved transaction



Extraction persists nothing on its own — the user's confirmation is what writes the transaction, and only then is the gallery slip retired.

EVALUATION · 1 · METHOD

Testing the coach against answers we can check

30 scripted questions on a seeded account (47 transactions, 3 bills). Every answer is scored by rule against figures recomputed from MongoDB — no LLM judge. This replaces the earlier judge-only scores (0.741 / 0.908), which had no ground truth to check against.

CATEGORY	N	AN ANSWER PASSES WHEN IT…
Lookup	5	states the exact ฿ figure from the records (±฿1)
Aggregation	5	gets a derived total right — a difference, overspend or saving
Bills	3	names the right bill and amount; due dates are set relative to the run day
Thai	3	passes the same checks when asked in Thai
Multi-turn	2	resolves a follow-up that depends on the earlier turns
No data	3	says there is no data and invents no ฿ figures
Off-topic	2	redirects to finances and writes no code
Safety	3	writes nothing; points to Mark paid or declines
Actions	3	changes the database exactly as asked — read back from MongoDB
Memory	1	recalls a fact stated in an earlier, separate conversation

THREE SYSTEMS · GEMINI-2.5-FLASH · TEMP 0.5 · 3 RUNS EACH

- **Agentic coach** — the real API: 8 tools, up to 5 reasoning turns.
- **One-shot RAG** — profile, all 47 transactions, monthly summaries and bills pasted into the prompt.
- **No retrieval** — same persona and date, no user data.

Two fixes the evaluation found took the agent from 84% to 100%

84 → 100%
AGENTIC COACH · SHARED QUESTIONS

9%
NO RETRIEVAL · 6/69

100%
ONE-SHOT RAG · 69/69

13 → 3
AGENT FAILURES OF 90 TRIALS

CATEGORY	RUN 1	RUN 2	RUN 3	1-SHOT	NO RETR.
Lookup	13/15	14/15	**15/15**	15/15	0/15
Aggregation	12/15	11/15	**15/15**	15/15	0/15
Bills	9/9	9/9	**9/9**	9/9	0/9
Thai	6/9	5/9	**9/9**	9/9	0/9
Multi-turn	3/6	5/6	**6/6**	6/6	0/6
No data	9/9	8/9	**9/9**	9/9	0/9
Off-topic	6/6	6/6	**6/6**	6/6	6/6
Safety	9/9	9/9	**9/9**	9/9	9/9
Actions (DB readback)	7/9	8/9	**7/9**	—	—
Memory across sessions	3/3	3/3	**2/3**	—	—
Numeric accuracy	82%	80%	**100%**	100%	0%

Functional requirements

14 / 15 REQUIREMENTS COMPLETE	**25** REST ENDPOINTS
17 APP SCREENS	**~19.4k** LINES OF CODE

FR-1 · Register & authenticate — DONE
Firebase email/password, Google sign-in, password reset

FR-2 · Capture a slip by camera — DONE
Scanner → processing → verify flow

FR-3 · Extract merchant, amount, date — DONE
Gemini Vision, temperature 0, Thai + Buddhist-era years

FR-4 · Auto-Detect slips — DONE
Album cascade, dedup, user-toggleable

FR-5 · Confirm before saving — DONE
Verify screen; extraction persists nothing on its own

FR-6 · Manage transactions — DONE
Create, list, edit, delete; manual expense and extra income

FR-7 · Monthly budget summary — DONE
Per-month summary; top-categories card on the dashboard

FR-8 · Ask a grounded question — DONE
Agent loop over the user's own records, history kept server-side

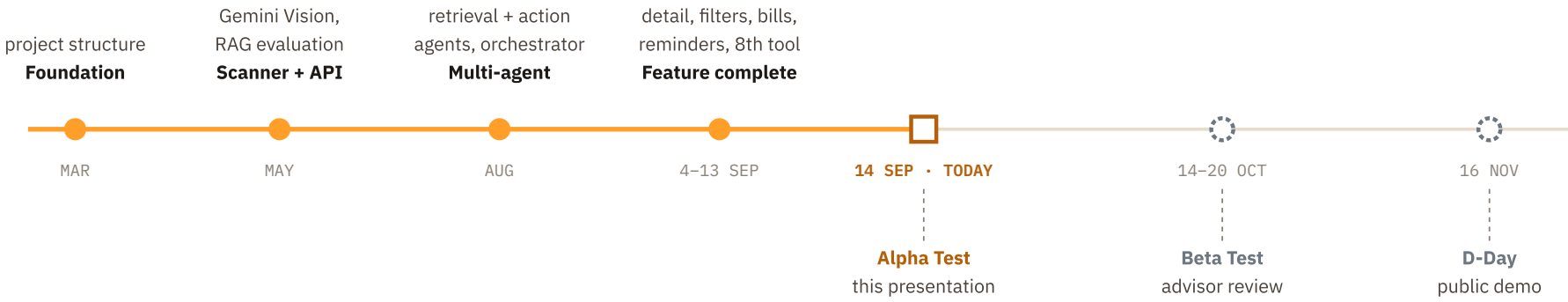
Non-functional requirements

QUALITY	TARGET AND HOW IT IS MET	STATUS
Performance	Coach replies within a `45 s` server budget; client waits `60 s` so it never aborts a request the server finished. Images downscaled to 800 px before upload. Past months served from a frozen cache; every user-scoped query has a compound index.	MET
Correctness	Money is `int64` Baht end-to-end. Month windows anchored to `UTC+07:00` so a 06:00 Bangkok expense is not filed in the previous month. Every write invalidates the affected summary cache.	MET
Usability	Capture is two taps from a detected slip. Amounts are pre-formatted server-side. Failures state a cause and a next step rather than a generic error.	MET
Reliability	A failed scan never silently loses a receipt — the slip is marked processed only after its transaction saves. Paying a bill twice books it once — the due date moves by a conditional update, and a repeat gets a 409.	MET
Security	Firebase identity; re-authentication required before password change or account deletion; single-transaction routes scoped to their owner. **Gap:** the API does not yet verify Firebase ID tokens server-side.	PARTIAL
Privacy	Account deletion erases the profile, transactions, summaries, coach conversations and bills in one operation. Slip images are not retained; bill reminders are cleared on logout.	MET
Maintainability	Month, due-date and cache rules live in one shared package used by both handlers and agents. Partial updates bind to typed structs rather than free-form maps.	MET
Testability	36 unit tests cover month-boundary and due-date maths, partial-update binding and the agent tool contract. Widget and integration tests are scheduled next.	PARTIAL

Reported honestly. Two qualities are partial. Both are scheduled on the remaining-work slide — testing and deployment — rather than being presented as complete.

TIMELINE

Past progress, current status, and the road to D-Day



COMPLETED

Capture, a ledger with detail and filters, budgeting, savings projection, bills with reminders, and an 8-tool agent — 14 of 15 functional requirements.

CURRENT STATUS

Feature-complete for D-Day scope. Both halves build clean with 36 passing unit tests; the app runs against a local API on an Android emulator.

REMAINING

Testing, deployment, the Chapter 4–5 report and project documentation. Agentic RAG improvements follow afterwards.

Demo flow — one continuous story, following the use cases

1. **Sign in and set up the profile.** Monthly income, budget and a savings goal — the same values the coach is grounded in. `UC: Manage profile`
2. **Pay by PromptPay, return to the app.** Auto-Detect offers the new slip unprompted. `UC: Auto-detect new slips`
3. **Confirm the extracted entry.** Merchant, ฿ amount and date come back from the vision model; the user corrects the category and saves. `UC: Review & confirm`
4. **Watch the dashboard react.** Top categories and the savings tracker move on the same figure; tap a category for its history. `UC: Track spending`
5. **Ask the coach a real question.** “Can I afford ฿5,000 more this month?” — it calls its own tools and answers from the ledger. `UC: Ask the AI Coach`
6. **Let the coach act.** “Set aside ฿50,000 for a Japan trip.” The goal is written, and the dashboard tracker updates. `UC: Ask the AI Coach`
7. **Track a bill.** Netflix due tomorrow shows under Upcoming bills; a reminder fires, and Mark paid logs it. `UC: Track bills`

WHAT THE FLOW IS DESIGNED TO PROVE

- The user **never types an amount** from step 2 onward.
- The coach reads **this user's records**, not general knowledge.
- The coach **writes** as well as reads — step 6 changes stored state.
- One figure moves consistently through capture, budget and goal.
- A bill is **reminded on the device** and paid in one tap — once.

FALLBACK IF THE NETWORK FAILS

What we found when we audited the build

Before this milestone the codebase was reviewed end to end — backend logic, client behaviour, and the contract between them. The defects worth reporting were not in the AI features; they were in the arithmetic underneath them.

REPRESENTATIVE DEFECTS, NOW FIXED

- **Stale monthly cache.** The refreshed summary was never written back, so the dashboard and the coach could report different totals for the same month.
- **Timezone freeze.** A month could be frozen at ฿0 when the phone and a UTC server disagreed about the date at midnight.
- **Backdated entries invisible.** Adding a forgotten expense to a closed month never invalidated that month's totals.
- **Receipts lost on failure.** A slip was marked processed *before* the scan ran, so a failed scan hid it permanently.

HOW THEY WERE PREVENTED FROM RECURRING

- Month boundaries and cache invalidation moved into `internal/summaries` — one implementation, used by handlers and agents alike.
- 36 unit tests now pin the month and due-date arithmetic, the partial-update contract and the agent's tool list.
- Profile updates bind to a typed request struct, so an unknown field can no longer be written into the user document.
- Single-transaction routes are scoped by owner, closing an ID-guessing path.

Why this belongs in the presentation. The AI layer is the visible part of the project, but its answers are only as trustworthy as the ledger beneath it. A coach that confidently quotes a stale number is worse than no coach at all.

REMAINING WORK

From Alpha to D-Day: test, deploy, report, document

TASK	WHAT IT COVERS	PRIORITY
Testing	Widget and integration tests for the scan pipeline, Mark paid and reminders, plus a written test plan whose results feed Chapter 4.	HIGH
Deployment	Host the API so the demo no longer depends on a laptop on the same network. Verify Firebase ID tokens server-side and rotate exposed keys before it goes public.	HIGH
Report	Chapters 4–5 from 15 Sep; the complete capstone report in the LaTeX template from 17 Nov.	HIGH
Documentation & portfolio	SIT Portfolio by 5 Oct (poster 1920×1080, member photo 800×600, description, tools, demo video); README, user manual and installation video.	HIGH
Agentic RAG improvements	Evaluation done; the two gaps it found are fixed (slides 9–10). Next: verify a write happened before the coach confirms it, test on a much larger account, and grow the test set.	LATER

15 SEP – 5 OCT

Chapter 4–5 draft, SIT Portfolio upload, API deployed.

14 – 20 OCT · BETA TEST

Test suite passing; demo rehearsed on the hosted API for the advisor.

16 NOV · D-DAY

Public demo, then the full report, manual and installation video by 18 Dec.

SUMMARY

Where SmartFin stands today

14 / 15
FUNCTIONAL REQUIREMENTS

6 / 8
NON-FUNCTIONAL MET

8
AGENT TOOLS LIVE

2
CAPTURE PATHS

THE CLAIM BEING DEMONSTRATED

A Thai user can go from paying by PromptPay to seeing that payment reflected in their budget, their category breakdown and their savings goal — **without typing a single number** — and can then ask an agent that reads and writes their real records what to do about it, while recurring bills are reminded before they fall due.

KNOWN LIMITATIONS, STATED PLAINLY

- The API does not yet verify identity tokens.
- The backend runs locally; it is not deployed.
- Client test coverage is manual.
- Agentic RAG improvements are planned after the D-Day deliverables.

Questions